

Flexibility, Scalability, and Modern Applications

 systemdesignschool.io/fundamentals/nosql-database



Introduction to NoSQL Databases

We explored SQL databases and their ACID guarantees. They excel at structured data with complex relationships. But they struggle with two modern problems: flexible schemas and horizontal scaling.

The Schema Rigidity Problem

A social media startup stores user profiles in PostgreSQL. Each user has a name, email, and bio. The schema looks clean:

```
CREATE TABLE users ( user_id int PRIMARY KEY, name varchar(255), email  
varchar(255), bio text );
```

Three months later, the product team wants verified badges for some users. Add a column. Six months later, premium users get custom themes. Add another column. A year in, influencers need follower counts, average engagement rates, and top hashtags. Every new feature requires a schema migration on a table with 10 million rows. Each migration locks the table. Deployments take hours.

Meanwhile, a competitor using MongoDB adds new fields instantly. No migration. No downtime. They ship features weekly while the SQL-based startup ships monthly.

The Horizontal Scaling Problem

An e-commerce site stores product catalogs in MySQL. Traffic grows. The database hits CPU limits. Vertical scaling helps temporarily: upgrade from 8 cores to 16, then 32, then 64. Eventually hardware limits appear. A 128-core server costs exponentially more than two 64-

core servers.

SQL databases struggle with horizontal scaling. Sharding requires manual partition key selection and application-level routing. Joins across shards become expensive. Foreign key constraints break. The team spends months building sharding logic instead of features.

NoSQL databases solve both problems. They embrace flexible schemas and distribute data automatically.

How NoSQL Works

NoSQL databases store data without fixed schemas. Different records can have different fields. Adding new fields requires no migration.

The term "NoSQL" means "Not Only SQL." These databases sacrifice some SQL guarantees for flexibility and scalability. They typically trade strong consistency for availability and partition tolerance.

Schema Flexibility

A document database stores user profiles as JSON:

```
{ "user_id": 123, "name": "Alice", "email": "alice@example.com", "bio":  
"Software engineer" }
```

Add a badge field for one user:

```
{ "user_id": 456, "name": "Bob", "email": "bob@example.com", "verified": true,  
"badge_color": "gold" }
```

No migration needed. The database accepts both documents. Application code handles missing fields. This flexibility enables rapid iteration.

Horizontal Scaling

NoSQL databases partition data automatically. DynamoDB distributes records across multiple servers based on partition keys. Add more servers to increase capacity. The database rebalances data automatically.

Cassandra replicates data across nodes with configurable consistency. Write to three replicas for durability. Read from one for speed. The system remains available even when nodes fail.

Types of NoSQL Databases

NoSQL databases optimize for different access patterns. Four main categories exist.

[Key-Value Stores](#) map keys to values. Redis stores session data. DynamoDB handles high-throughput lookups. These databases excel at simple get/put operations with predictable performance.

Document Databases store JSON-like documents. MongoDB manages product catalogs. CouchDB handles content management. Documents can have nested fields and arrays, matching application object models.

Column-Family Stores organize data by columns instead of rows. Cassandra handles time-series data. HBase processes analytics workloads. Scanning specific columns becomes efficient while full-row scans remain expensive.

Graph Databases model relationships explicitly. Neo4j powers social networks. ArangoDB supports recommendation engines. Traversing relationships (friend-of-friend queries) becomes fast while aggregations remain slow.

Each type trades off different characteristics. Choose based on access patterns, not just data structure.

When to Use NoSQL

NoSQL databases fit specific scenarios where SQL struggles.

Unpredictable schemas benefit from NoSQL flexibility. [Instagram's user profiles](#) evolved rapidly during growth. Adding profile fields weekly would cripple MySQL migrations. Cassandra absorbed changes without downtime.

High write throughput demands horizontal scaling. [Discord](#) stores trillions of messages. A single database can't handle millions of writes per second. Cassandra partitions writes across hundreds of nodes.

Simple access patterns don't need SQL complexity. Shopping carts require key-value lookups by session ID. Session data needs no joins or transactions. Redis provides microsecond latency without SQL overhead.

Global distribution benefits from eventual consistency. Social media feeds can tolerate stale data for seconds. Users don't notice if a like count updates with slight delay. DynamoDB replicates globally with configurable consistency.

Large unstructured datasets exceed SQL capabilities. Log aggregation systems collect gigabytes per second. Elasticsearch indexes and searches text without schemas. SQL full-text search becomes impractical at this scale.

Challenges with NoSQL

NoSQL databases sacrifice guarantees that SQL provides.

Eventual consistency complicates application logic. Two users editing the same document might see different versions. Application code must handle conflicts. [Dynamo's shopping cart](#) merges conflicting updates by keeping all items ever added. Users occasionally see deleted items reappear.

No ACID transactions across records forces workarounds. Transferring money between accounts requires two separate updates. If the system fails between updates, money disappears or duplicates. Applications implement saga patterns or accept inconsistency.

Limited query capabilities require denormalization. SQL joins multiple tables efficiently. NoSQL databases store redundant data to avoid joins. A user's posts include their name in each document. Name changes require updating thousands of posts.

Operational complexity increases. SQL databases mature tooling exists for backup, monitoring, and debugging. NoSQL databases require learning new tools. Cassandra's eventual consistency makes debugging difficult. DynamoDB's pricing model requires careful capacity planning.

Choosing Between SQL and NoSQL

Most applications can use either database type. Start with the simpler choice. [PostgreSQL handles 99% of use cases](#) including JSON fields and horizontal scaling extensions.

Use NoSQL when specific requirements demand it. Need millions of writes per second? Use Cassandra. Schema changes deploy daily? Use MongoDB. Simple key-value lookups only? Use Redis.

Many systems use both. E-commerce sites store orders in PostgreSQL for ACID guarantees, cache sessions in Redis for speed, and index products in Elasticsearch for search. Choose the right tool for each component.

We cover specific NoSQL types in detail next: [key-value databases](#), [document databases](#), and [full-text search](#).